

Module 3: Binary File Handling (Byte-Based) - Complete Study Notes

Welcome to **Module 3** of our Java File Handling course! Now that you've mastered text handling in Module 2, we shift to **binary files**—non-text data like images (.jpg), executables (.exe), PDFs, or serialized objects. Binary handling uses byte streams (`InputStream` / `OutputStream`) to read/write raw bytes, preserving exact data without interpretation (no auto-decoding like text).

By the end of this module, you'll:

- Write and read binary data using basic and buffered byte streams.
- Copy entire binary files efficiently.
- Build utilities for binary operations with error handling.
- Understand when to use binary vs. text (e.g., detect via extension).

Key Concepts:

- **Byte Streams vs. Character Streams:** Bytes are 8-bit raw data; no Unicode handling—perfect for binaries, risky for text (can corrupt).
- **Buffering:** `BufferedInputStream` / `BufferedOutputStream` for speed on large files.
- **Chunked Reading:** Use buffers (e.g., 4KB) to avoid loading huge files into memory.
- **IOException:** Essential for disk/network errors.

Prerequisites: Modules 1 & 2. Use `java.io` package. Test with a sample binary like a small .jpg.

To visualize the byte stream hierarchy, here's a diagram:

1. Core Concepts: Byte Streams Overview

Byte streams treat data as a sequence of `byte` (8-bit). Key classes:

- **Abstract Base:** `InputStream` (read), `OutputStream` (write).
- **File-Specific:** `FileInputStream` / `FileOutputStream` —direct file access.
- **Buffered:** `BufferedInputStream` / `BufferedOutputStream` —cache data for efficiency.
- **Why Bytes?** Exact preservation; use for anything non-UTF8.

Import Statements (add at top):

```
import java.io.*; // Includes InputStream, OutputStream, etc.
```

2. Step-by-Step Code Examples with Comments

Examples are runnable classes. Use a "data" folder; create a sample binary (e.g., download a tiny .png or use byte array).

Step 1: Writing Binary Data

`FileOutputStream` writes raw bytes. Example: Simulate writing image data from a byte array.

Sample Code: BinaryWriterBasic.java

```
public class BinaryWriterBasic {
    public static void main(String[] args) {
        String filename = "data/sample_binary.dat"; // Output file—any extension

        // Sample binary data: ASCII "Hello" as bytes (could be image bytes)
        byte[] data = {72, 101, 108, 108, 111}; // H=72, e=101, etc. (or read from file)

        try (FileOutputStream fos = new FileOutputStream(filename)) { // Creates/overwrites file
            // Write entire array at once
            fos.write(data); // Flushes bytes to disk

            // Alternative: Write partial—fos.write(data, 0, 3); for first 3 bytes
            System.out.println("Binary data written: " + data.length + " bytes to " + filename);
        } catch (IOException e) {
            // Handles: Permission denied, disk full
            System.err.println("Write error: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```

Output: Creates "sample_binary.dat" (5 bytes). View in hex editor to see 48 65 6C 6C 6F.

Buffered Version (For larger data):

Sample Code: BinaryWriterBuffered.java

```
public class BinaryWriterBuffered {
    public static void main(String[] args) {
        String filename = "data/large_binary.dat";
        byte[] largeData = new byte[10000]; // Simulate large binary (e.g., image)
        // Fill with dummy data: for(int i=0; i<largeData.length; i++) largeData[i] = (byte)i;

        try (BufferedOutputStream bos = new BufferedOutputStream(new FileOutputStream(filename)))
            // Buffer collects writes, flushes in chunks (default 8KB)
            bos.write(largeData); // Efficient for big arrays

            bos.flush(); // Optional: Force write before close
            System.out.println("Buffered binary written: " + largeData.length + " bytes");
        } catch (IOException e) {
            System.err.println("Buffered write error: " + e.getMessage());
            e.printStackTrace();
        }
    }
}
```



Why Buffer? Reduces OS calls—key for MB-sized files.

Step 2: Reading Binary Data

`FileInputStream` reads into byte arrays. Use loop for large files.

Sample Code: BinaryReaderBasic.java (Reads "sample_binary.dat" from Step 1)

```

public class BinaryReaderBasic {
    public static void main(String[] args) {
        String filename = "data/sample_binary.dat";

        try (FileInputStream fis = new FileInputStream(filename)) {
            // Read entire file into byte array (for small files only)
            byte[] buffer = new byte[(int) new File(filename).length()]; // Size from File.length()
            int bytesRead = fis.read(buffer); // read() fills buffer, returns count or -1 for EOF

            if (bytesRead == -1) {
                System.out.println("File empty!");
            } else {
                // Print as hex for binary visibility
                System.out.print("Read bytes (hex): ");
                for (byte b : buffer) {
                    System.out.printf("%02X ", b); // %02X pads to 2 digits
                }
                System.out.println("\nTotal: " + bytesRead + " bytes");
            }
        } catch (IOException e) {
            System.err.println("Read error: " + e.getMessage());
            e.printStackTrace();
        }
    }
}

```

Output:

```

Read bytes (hex): 48 65 6C 6C 6F
Total: 5 bytes

```

Buffered/Chunks Version (For large files—don't load all at once):

Sample Code: BinaryReaderBuffered.java

```

public class BinaryReaderBuffered {
    public static void main(String[] args) {
        String filename = "data/large_binary.dat";

        try (BufferedInputStream bis = new BufferedInputStream(new FileInputStream(filename))) {
            byte[] buffer = new byte[1024]; // 1KB chunk—adjust for needs
            int totalBytes = 0;
            int bytesRead;

            System.out.println("Reading in chunks:");
            while ((bytesRead = bis.read(buffer)) != -1) { // Loop: read fills buffer up to size
                totalBytes += bytesRead; // Accumulate total
                // Process chunk: e.g., print first byte of each
                if (bytesRead > 0) {
                    System.out.println("Chunk: " + bytesRead + " bytes, first: " + buffer[0]);
                }
            }
            System.out.println("Total read: " + totalBytes + " bytes");
        } catch (IOException e) {
            System.err.println("Buffered read error: " + e.getMessage());
            e.printStackTrace();
        }
    }
}

```

Why Chunks? Memory-safe for GB files—process as you read.

Step 3: Copying Binary Files

Combine read/write for duplication (e.g., backup .jpg).

Sample Code: BinaryCopier.java (Copies "input.jpg" to "output.jpg"—provide input)

```

public class BinaryCopier {
    public static void main(String[] args) {
        String source = "data/input.jpg"; // Assume exists
        String dest = "data/output.jpg";

        try (FileInputStream fis = new FileInputStream(source);
            FileOutputStream fos = new FileOutputStream(dest)) {

            byte[] buffer = new byte[4096]; // 4KB—good balance
            int bytesRead;

            // Loop: Read chunk, write exact amount read
            while ((bytesRead = fis.read(buffer)) != -1) {
                fos.write(buffer, 0, bytesRead); // 0=start offset, bytesRead=length
                fos.flush(); // Optional for progress
            }
            System.out.println("File copied successfully!");
        } catch (IOException e) {
            System.err.println("Copy error: " + e.getMessage());
            e.printStackTrace();
        }
    }
}

```

Result: Identical "output.jpg"—verify sizes match.

Buffered Copy (Faster):

Replace with `BufferedInputStream` / `BufferedOutputStream` for even better perf.

3. Practice: Building Reusable Methods

Create utilities for binary tasks.

Classwork 1: Binary Utility

Methods for write/read bytes.

Complete Class: BinaryUtil.java

```

import java.io.*;

public class BinaryUtil {
    /**
     * Writes byte array to file.
     * @param filename Output path
     * @param data Bytes to write
     */
    public static void writeBytes(String filename, byte[] data) {
        try (FileOutputStream fos = new FileOutputStream(filename)) {
            fos.write(data); // Simple write
            System.out.println("Wrote " + data.length + " bytes to " + filename);
        } catch (IOException e) {
            System.err.println("Write bytes error: " + e.getMessage());
        }
    }

    /**
     * Reads entire file into byte array (for small files <1MB).
     * @param filename Input path
     * @return Byte array, or null on error
     */
    public static byte[] readBytes(String filename) {
        File file = new File(filename);
        byte[] data = new byte[(int) file.length()];
        try (FileInputStream fis = new FileInputStream(file)) {
            int bytesRead = fis.read(data); // Assume full read
            if (bytesRead == data.length) {
                System.out.println("Read " + data.length + " bytes from " + filename);
                return data;
            } else {
                System.err.println("Incomplete read!");
            }
        } catch (IOException e) {
            System.err.println("Read bytes error: " + e.getMessage());
        }
        return null;
    }

    public static void main(String[] args) {
        // Test: Write sample
        byte[] sample = {0x01, 0x02, 0xFF}; // Binary values
        writeBytes("data/test.dat", sample);
    }
}

```

```
// Read back
byte[] read = readBytes("data/test.dat");
if (read != null) {
    for (byte b : read) {
        System.out.printf("0x%02X ", b); // Hex print
    }
}
}
```

Sample Output:

```
Wrote 3 bytes to data/test.dat
Read 3 bytes from data/test.dat
0x01 0x02 0xFF
```

Depth Adder: Hex Dumper (Visualize Binary)

For debugging—prints file as hex dump.

Enhanced Class: HexDumper.java


```

import java.io.*;

public class HexDumper {
    /**
     * Dumps file contents as hex (e.g., for images).
     * @param filename Path to dump
     * @param chunkSize Bytes per line
     */
    public static void dumpHex(String filename, int chunkSize) {
        try (FileInputStream fis = new FileInputStream(filename)) {
            byte[] buffer = new byte[chunkSize];
            int bytesRead;
            int lineNum = 0;

            System.out.println("Hex Dump of " + filename + ":");
            while ((bytesRead = fis.read(buffer)) != -1) {
                System.out.print(String.format("%04d: ", lineNum * chunkSize)); // Line offset
                for (int i = 0; i < bytesRead; i++) {
                    System.out.printf("%02X ", buffer[i]); // Hex bytes
                }
                System.out.println(); // New line
                lineNum++;
            }
        } catch (IOException e) {
            System.err.println("Dump error: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        dumpHex("data/sample_binary.dat", 5); // 5 bytes/line
    }
}

```

Output (for "Hello"):

Hex Dump of data/sample_binary.dat:
 0000: 48 65 6C 6C 6F

Why? Essential for verifying binary integrity.

4. Tasks and Exercises

Hands-on! Use binary files (e.g., copy a small photo).

Task 1: Basic Binary Writer (Beginner - 10 mins)

- Class: `ByteGenerator` .
- Method: `generateSample(String filename, int size)` —fill `byte[]` with sequential values (0 to size-1), write it.
- Test: Generate 100 bytes, read back with `readBytes` , check first/last.
- **Expected:** Handle size=0 (empty file).

Task 2: File Comparer (Intermediate - 15 mins)

- Class: `BinaryComparer` .
- Method: `compareFiles(String file1, String file2)` —read both, check byte equality; print diff offset if mismatch.
- Test: Copy a binary, alter one byte, compare.
- **Challenge:** Use chunked read for large files.

Task 3: Image Resizer Stub (Advanced - 20 mins)

- Extend `BinaryCopier` : Add `resizeCopy(String src, String dest, double factor)` —but just truncate bytes (e.g., read `factor*length`).
- Bonus: Use `DataOutputStream` to write structured data (e.g., int header + bytes).
- **Why?** Simulates real binary processing.

Task 4: Error Handler (Debugging - 10 mins)

- Modify `BinaryCopier` for `FileNotFoundException` : Print "Source missing—create it!" and skip.
- Test: Bad source path—no crash.

Grading Rubric (Self-Assess):

- Code compiles/runs: 40%
- Comments on loops/buffers: 30%
- Chunked for large files: 20%
- Hex/debug output: 10%

5. Common Pitfalls and Best Practices

Pitfall	Fix
Treating binary as text	Use hex viewers (e.g., HxD)—don't open in Notepad
OutOfMemory on big reads	Always chunk: <code>while((bytesRead = read(buffer)) != -1)</code>
Incomplete writes	Check return value of <code>write()</code> or use <code>flush()</code>
Platform byte order	Use <code>ByteOrder</code> in NIO for cross-platform (advanced)
Forgetting to cast bytes	Bytes are signed (-128 to 127)—use unsigned: <code>(b & 0xFF)</code>

Pro Tip: For objects, use `ObjectInputStream` (Module 4). Detect type: if `endsWith(".txt")` use text, else binary.

Resources:

- Oracle Docs: [FileInputStream](#)
- Practice: Copy a video file—time buffered vs. non.

Next Up: Module 4 (Advanced). Crush these tasks—code review?